When does a manufacturability term change what evolution selects? Measurement, repair and a computable screening bound, with the ForgeCraft platform

Zhenxin Liu[1]*, Open Researcher and Contributor ID (ORCID): 0009-0005-0773-647X

[1] Beijing Jiaotong University, Beijing, China

* *Corresponding author. E-mail: liuzhenxin0611@qq.com*

Abstract

Background. Morphology–control co-design searches simulated link-and-joint assemblies, so the part geometry, tolerances and clearances that decide whether a design can be built never enter the objective. Adding a manufacturability term is the usual remedy, and its effectiveness is usually assumed rather than measured. **Aim.** We ask when such a term can change what selection keeps, and we answer it with measurement rather than with a new search algorithm. The vehicle is ForgeCraft, an open-source Python platform that couples declarative part catalogues, graph neural network (GNN) co-evolution with proximal policy optimisation (PPO) controllers, MuJoCo evaluation, and an export layer emitting tessellated assemblies in the Standard for the Exchange of Product model data (STEP), application protocol 242 (AP242), drawings in Drawing Exchange Format (DXF), interference reports and bills of materials. **Method.** We ablate the term counterfactually within single runs, on two instrument builds, and derive a bound on the reordering any weight can produce. **Results.** The shipped build (tag v0.4.0) could not act on the term at all: the score's range lay inside [0.80, 1.00] and its weight was never read from the task file, so every run silently used 0.05. After repair the score's mean within-catalogue range rises from 0.024 to 0.225, yet discordant pairs remain at or near zero, because a pair can reverse only when its fitness gap is smaller than the term's contribution. The fraction of such pairs is computable in one generation and bounds reordering from above; it is 6–8% for one catalogue and 12–48% for another over a twentyfold weight sweep, which is why the same nominal weight behaves differently in each. Whether crossing that bound improves populations is not resolvable at three seeds; a power analysis requires 13–143. **Limitation.** No evolved design has been fabricated, and the export layer reveals defects the objective never sees: every final individual contains colliding part pairs.

Keywords: evolutionary robotics · morphology optimisation · design for additive manufacturing · STEP AP242 · reproducibility · open-source software

1 Introduction

The body of a robot largely determines what that robot can do, and in practice the body is normally fixed by human designers before any controller is learned. Co-design — the joint optimisation of morphology and control — has been studied since the first work on evolving virtual creatures [1] and has been revisited repeatedly with modern machinery: graph-grammar design spaces [2], learned transform-and-control policies [4], self-assembling modular morphologies [5], and quality-diversity search over collections of designs [6, 7].

A persistent limitation runs through this literature. Evaluated designs are almost always *simulated* bodies: abstract link-and-joint assemblies whose geometric realisation is left to the reader. An evolved morphology cannot be built, however, without answering questions the simulation never asks. Which abstract links correspond to printable parts? Do they fit a build volume? Are the walls thick enough? Do adjacent parts collide once real bearings, fasteners and clearances are introduced? Can the joint be assembled at all, and what does the bill of materials cost? These are not cosmetic concerns: they decide whether an evolved design leaves the computer. When manufacturability is absent from the objective, search is free to exploit shapes that are optimal in simulation and unbuildable in reality, and a post-hoc buildability filter then discards exactly the designs the search converged on.

Two further gaps compound the problem. **Reproducibility:** morphology-evolution results are frequently reported for bespoke codebases that cannot be re-run, so comparisons between methods remain approximate. **Fragmentation:** the capabilities required for a complete study — morphology generation, physics evaluation, control learning, geometric modelling and manufacturing preparation — are normally distributed across separate tools with incompatible data models, so every stage boundary is crossed by hand.

Contribution. The measurement is the contribution; the platform is what made it possible. (i) A counterfactual method that removes an objective term *within* a single run, so that no re-evaluation is needed and unpaired worker seeds cannot contaminate the comparison. (ii) A two-build measurement showing that the shipped release could not act on its manufacturing term at all, and identifying the two instrumentation defects responsible — a threshold-triggered score with no gradient, and a weight lookup that read the wrong configuration namespace, so every task silently used a hard-coded fallback. (iii) A bound, proved in Section 7.5 and verified on every cell of two sweeps, on how many pairs of individuals any weight can reorder, together with the single-generation quantity that makes the bound usable before a long run is spent: the fraction of pairs whose fitness gap is smaller than the term's contribution. (iv) A replication budget — 13 to 143 seeds — that states

what evidence a population-level claim would require. The platform itself contributes an explicit parametric representation of evolved bodies as mechanical parts sharing parameters with the bodies that were evaluated, and an export chain that converts such a body into the deliverables a fabricator can consume. We also report, in Section 8, the defects and inconsistencies found in our own release, because a framework making manufacturing claims is only as useful as its account of where those claims currently stop.

2 Related work and the position of this work

Modern co-design systems encode a body as a graph and optimise it with evolutionary search while a learned controller is fitted to each candidate; ForgeCraft follows that template and differs in the treatment of the output. The body graph is not only a simulation input but also the specification from which concrete mechanical geometry is generated, so the artefact evaluated and the artefact exported derive from one shared parameter set (Section 3.4).

Three platforms are close enough in ambition to warrant direct comparison, and Table 1 summarises it. **Evolution Gym** [8] is a large-scale benchmark for co-optimising the design and control of soft robots, in which a body is a 2D lattice of material cells simulated as a mass-spring system. **ARIEL** (https://ci-group.github.io/ariel/), the successor to the Revolve simulator [9], covers genome encoding, evolution, MuJoCo simulation and result analysis for modular robots. **JBotEvolver** [10] is a long-established Java platform for evolutionary robotics research and teaching, built around a 2D differential-drive engine.

All three are strong at what they set out to do, and ForgeCraft integrates the same class of components rather than reimplementing them. The relevant difference is the terminal stage: in the released feature sets of all three, the pipeline ends once a simulation result has been produced and logged, and none describes a route from an evolved body to manufacturing deliverables. For Evolution Gym this follows from the representation itself — a 2D voxel matrix has no corresponding fabrication output. For ARIEL and JBotEvolver, whose robots are built from physical modules or wheeled bases, the morphologies are realisable in hardware by construction, but the mapping from an evolved graph to machine-readable part geometry, drawings and bills of materials is not part of the framework.

Design for additive manufacturing [11] and design for manufacture and assembly [12] supply the criteria a buildable part must satisfy — minimum feature size, wall thickness, build-volume fit, support and orientation, assembly complexity — and product data exchange between computer-aided design (CAD) systems is standardised in International Organization for Standardization (ISO) 10303, whose AP242 part [13] covers managed model-based 3D engineering, with general tolerances given by ISO 2768-1 [14]. This body of work is mature, but it does not address a design generated by a search free to exploit any unmodelled degree of freedom — precisely the case ForgeCraft is built for.

Platform	**Morphology representation**	**Co-design search**	**Physics**	**Manufacturing deliverables**	**Reference**
Evolution Gym	2D lattice of material voxels (soft / rigid / actuator)	Yes — design and control co-optimisation; task suite and baseline algorithms	Own C++ mass-spring engine	None in the released benchmark	Bhatia et al. [8]
ARIEL (formerly Revolve)	Modular robot graph decoded from NDE / compositional pattern-producing network (CPPN) / tree / vector genotypes	Yes — body–brain co-design with evolutionary-algorithm (EA) operators	MuJoCo	None in the published feature set	[9]
JBotEvolver	2D differential-drive kinematics; controller evolution on a fixed or scripted body	Controller evolution; body variation is not the focus	Own 2D engine	None	[10]
ForgeCraft	Typed graph of parametric parts drawn from declarative YAML catalogues	Yes — NSGA-II/III and Multi-dimensional Archive of Phenotypic Elites (MAP-Elites) over bodies; PPO controller conditioned on a GNN body embedding	MuJoCo; optional Genesis backend	**Tessellated STEP AP242 assembly, tolerance-annotated DXF drawings, interference report, bill of materials, manufacturability score, production archive**	This paper

Table 1 Feature comparison against three widely used evolutionary-robotics platforms. The manufacturing column records what each project's released documentation and repository describe at the time of writing; it does not rule out unpublished extensions. Rows are ordered by breadth of published scope.

3 System overview

ForgeCraft is a layered Python package of 192 modules (approximately 60,000 lines) with a deliberately narrow interface between layers, so that individual research questions can be addressed without modifying the rest of the stack.

3.1 Core morphology representation

The central data structure is `MechanicalBody`, a directed graph of `Part` and `Joint` objects. Parts carry a type and a parameter dictionary, so a body is a typed graph rather than a fixed topology; the representation is memory-conscious by design (`__slots__` on both classes) because populations of thousands of bodies are held in memory during search. Parts are drawn from a *catalogue*: a declarative YAML description of the available part types with their geometric primitive, parameter ranges, mass, friction and actuation properties. Because the catalogue is data rather than code, a new design space is a configuration file rather than a fork; 21 catalogues ship with the release, spanning primitive parts, legged robots, grippers, hoppers and parameterised part libraries.

3.2 Physics evaluation

Evolved bodies are compiled to MuJoCo models in the MuJoCo Extensible Markup Language (XML) format (MJCF) for rigid-body simulation, with an optional Genesis backend for GPU-accelerated evaluation. To keep search tractable, the evaluation path uses deliberately simplified collision geometry (convex proxies), which decouples the cost of physics evaluation from the cost of geometric detail. Environments follow the Gymnasium API [15], and task specifications — reward composition, episode budget, termination criteria — are themselves declarative YAML.

One modelling detail determines whether the reward terms carry any signal at all: the MJCF builder attaches a `<freejoint/>` to the root body, so every compiled model has a full six-degree-of-freedom floating base. An earlier revision omitted that element and welded the root to the world frame, which made the displacement-, speed- and fall-based reward terms identically zero. Section 6 records the measured before-and-after effect of the repair.

3.3 Search

Co-evolution alternates between population-level search over morphologies and reinforcement-learning updates of the controllers conditioned on those morphologies. Morphologies are encoded by a message-passing graph neural network [16, 17] into fixed-length embeddings, and controllers are PPO [18] actors conditioned on those embeddings. Conditioning the policy on a morphology embedding allows a controller to be trained for an arbitrary topology, but ForgeCraft trains a *separate* controller per evaluated individual within that individual's evaluation budget, optionally warm-started from the previous generation: it does not learn a single policy that generalises zero-shot across body topologies, and we make no such claim. Selection combines elitism, tournament and Pareto-based multi-objective selection (NSGA-II/III [19, 20]) while a MAP-Elites archive [6] maintains behavioural diversity. Search is made affordable by adaptive evaluation budgets that grow with the generation index, and runs support checkpoint and resume.

3.4 Geometry and manufacturing layer

A parametric generator produces concrete mechanical geometry for each abstract part — chassis with ribs and counterbores, motors with cooling fins and end bells, bearings with shields, tubes with threaded ends, springs, feet and battery packs — from the same parameter dictionary used during search. Nine classes of part are generated at four vertex-density presets, and parts receive physically based rendering materials from ten presets. Because search-time bodies and export-time geometry share parameters, the exported artefacts correspond to the design that was actually evaluated. Optional modules provide finite-element and isogeometric analysis, additive-manufacturing pre-processing, knowledge storage and LLM-assisted design agents, isolated behind optional installation extras so that the core pipeline keeps a small dependency footprint; Section 8 records that these analysis modules are incomplete and unused in every result reported here.

4 The manufacturing-aware export chain

The export chain is the contribution claimed in this paper. A single entry point, `export_all_enhanced`, takes an evolved body plus the catalogue it was drawn from and performs the following stages end to end:

1. **Parametric solid generation** for every part in the evolved body.
2. **Connection features** — bolt holes, hinge bearing seats and snap fits generated from the joint type, with clearances derived from fused deposition modelling (FDM) tolerance profiles.
3. **Tolerance and shrinkage compensation** for the selected print profile.
4. **Support and orientation planning**, with support-volume reporting.
5. **Interference checking** between mating part pairs, classifying each fit as clearance, transition or interference.

6. **STEP AP242 assembly export** for CAD interchange, subject to the qualification below.
7. **2D engineering drawings** in DXF with tolerance annotation.
8. **Bill of materials** with mass roll-up and material recommendations.
9. **Manufacturability scoring**, decomposed into build-volume, wall-thickness, assembly-complexity, torque-budget and cantilever checks.
10. **Production packaging** into one archive that can be handed to a fabricator, together with a Unified Robot Description Format (URDF) file for downstream robotics tooling and an optional export of the trained policy to the Open Neural Network Exchange (ONNX) format.

Two properties matter more than the stage list. First, the manufacturability score described in stage 9 is wired into the search layer — a manufacturability term and a minimum-score constraint are both available per task — and Section 7 measures what that wiring currently achieves. Second, the STEP output carries a qualification we will not bury: the exported entities are tessellated (`TESSELLATED_ITEM` with `COORDINATES_LIST` and `TRIANGULATED_FACE`, under the `AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF` schema). The file is valid AP242 and readable by tessellation-aware viewers, but it carries triangle meshes rather than analytic non-uniform rational B-spline (NURBS) surfaces and topological B-rep edges, so it cannot be edited parametrically on import. A fabricator receiving this package would typically rebuild parametric features from the DXF drawings and the bill of materials rather than from the solid. Closing that gap is future work.

5 Worked example: one complete run

The workflow is exercised by an example committed to the repository (`examples/demo_robot/`), recording the artefacts of a finished evolution run (generation 89, individual 005), started with a single command:

```
python -m forgecraft.main --catalog speedster --task speed --population 32 --generations 100 --seed 42 --export --export-dir design_output
```

The command-line interface (CLI) `--export` flag drives the lightweight export path (stereolithography (STL) and STEP meshes, 3MF, URDF, a bill of materials (BOM) and a review report). The fuller presentation chain, which additionally produces DXF drawings, glTF scenes, rendered views and the packed production archive, is `export_all_enhanced` and is reached through the Python API.

The recorded design comprises 25 parts and 24 joints with a stored composite fitness of 12954.87, dominated by movement terms (`displacement` = 10452.36, `speed` = 2497.80, `upright` = 46.41). The body has eight actuated joints, so the review report no longer emits the "no actuated joints, the body cannot move under its own power" warning that the previous snapshot carried. We nevertheless report the manufacturing diagnostics in full, because they say more about the state of the release than the fitness value does (Table 2, Fig. 1).

Quantity	Value	Source
Parts / joints	25 / 24	`best_body.json`
Composite fitness	12954.87	`evolution_history.json`
Speed component	2497.80	`evolution_history.json`
Displacement component	10452.36	`evolution_history.json`
Upright component	46.41	`evolution_history.json`
Actuated joints	8	`best_body.json`
Colliding part pairs	120	interference check
Collapsed joint ranges	6 of 24	joint-range warnings
Implausible joint ranges	5 of 24 (widest 888.18 rad)	joint-range warnings
Manufacturing aggregates	`overall` 0.9925; score 0.70	`manufacturability.json`
Mass, three code paths	5.244 / 6.233 / 7.005 kg	`bom.json`; review report
Cost, three code paths	157.31 / 442.11 / 255.13 USD	`bom.json`; review report
Print estimate	85.0 h, 2862 g PLA	review report

Table 2 Diagnostics for the reference run. The mass, cost and aggregate rows are not defects of the pipeline but of its bookkeeping: the same body is measured differently by different code paths, and nothing reconciles them.

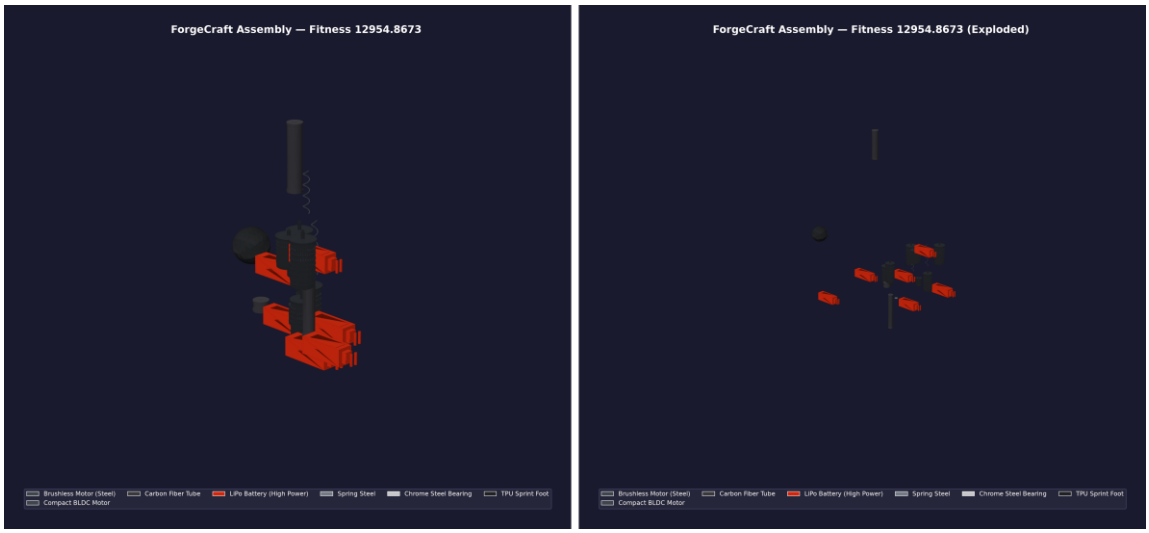


Fig. 1 Assembled (left) and exploded (right) views of the evolved body of Section 5, rendered by the framework's lightweight rasteriser. The 25 parametric parts shown are the ones from which the STEP assembly, DXF drawings and bill of materials are produced. The body is a prototype produced by the pipeline, not a validated design: it contains 120 colliding part pairs and eleven joints with implausible parameter ranges (Table 2).

Three caveats follow, and we state them rather than omit them. First, a stored fitness of 12954.87 is **not** evidence of a high-performing robot: it is a weighted sum dominated by movement terms, reached while the body's parts interpenetrate in 120 pairs and its joints admit physically meaningless travel, neither of which the objective penalises. The honest reading is that the objective now carries a locomotion signal — which, before the repair described in Section 6, it did not — and that self-collision and joint-limit terms must be added before evolved bodies can be read as engineering designs. Second, the aggregate figures disagree: `overall` (0.9925) and `manufacturability_score` (0.70) are two distinct definitions rather than an arithmetic error (Section 7), while the mass of this single body is reported as 5.244 kg, 6.233 kg and 7.005 kg by three sites, and cost as 157.31 against 442.11 in United States (US) dollars (USD). Each site derives volume and unit cost from a different formula — bounding-box extents, per-part parameters, or a separate price table — and we report all of these figures rather than the most flattering one. Third, the committed snapshot is a stored artefact: STL, STEP, DXF, URDF and the packed archive regenerate from `best_body.json` and are deliberately not committed, to keep the repository small.

We therefore present this example as evidence for three narrow claims only: that the pipeline runs end to end from a task specification to a production package, that evolutionary search now receives a locomotion signal, and that the export chain emits inspectable, machine-readable deliverables. It is not evidence that the framework discovered a high-performing robot.

6 Verification and reproducibility

The repository contains 424 automated tests — 253 in the core suite and 171 module-level — covering morphology graph invariants, catalogue loading, genetic operators, selection schemes, checkpoint round-trip, physics model construction, geometry generation, tolerance and connector calculation, interference checking and export formats. Continuous integration runs the suite on Python 3.10–3.12 at every commit, skipping GPU-dependent tests when no Compute Unified Device Architecture (CUDA) device is present. Runs are seeded, task and catalogue definitions are declarative, and checkpoints capture both evolutionary and training state, so an interrupted run resumes to an identical configuration.

The most consequential defect found while preparing this release illustrates why a green test suite is not sufficient evidence. In the previous revision, `forgecraft/simulation/builder.py` emitted the root body without a `<freejoint/>`, so a six-part, five-joint body compiled with no floating base: the centre-of-mass sensor was constant, termination criteria of the form `_com_z < fall_height` never fired, and the `displacement`, `speed` and `velocity` reward terms were identically zero, removing the locomotion signal from the objective altogether. The repair is verified by measurement rather than assertion: the same body now compiles to `nq = 9, nv = 8`, with the free joint at `qposadr = 0` and hinges at `qposadr = 7` and `8`; in a 300-step zero-action rollout the centre of mass descends from 0.476 m to 0.191 m, translates 0.064 m along x and fires the fall termination at step 299 — all exactly zero before the repair. Two latent Gymnasium-contract errors in `forgecraft/rl/env.py` were exposed by the same investigation and are fixed with it: `step()` returned `np.bool_` rather than `bool`, and `reset()` perturbed `qpos` by joint index, which with a floating base corrupts the base position and quaternion.

7 What a manufacturing term actually does to selection

The framework's design goal is that buildability is optimised rather than filtered. We tested that claim on two instrument builds of the same codebase. The shipped build (tag v0.4.0) returned a clean negative; investigating why exposed two instrumentation defects, which we report and repair. The repaired build (commit a246c14) returns a different answer: the term can reorder a population only to the extent that individuals are separated by fitness gaps smaller than its own contribution, and that extent is measurable in one generation. What the repaired build does not return is a population-level improvement, and Section 7.7 states the replication that would be needed to look for one.

7.1 Removing the term without re-running

Comparing two independently evolved arms is unreliable in this codebase: evaluation runs in worker processes, and on Windows those workers are spawned rather than forked, so they do not inherit the parent seed and two same-seed arms are not strictly paired. We therefore measured *within* a single run. Each individual carries a fitness that includes the manufacturability term and a directly measurable value of that term, so the counterfactual fitness obtained by removing it is `fitness_without = fitness_with − w_mfg · mfg`, with no re-running required. The effective weight is `w_mfg = 0.05`: the task file lists `manufacturability` among its fitness components but declares no matching reward component, so `get_weight` returns zero and the aggregator's fallback applies (defect D2 in Section 7.3). Every statistic in this section is recomputed from the stored final populations by `recompute_tables.py` in the ablation experiment directory, which reads the same JSON files the runs wrote.

7.2 First measurement (v0.4.0): the term is inert

The term does not change selection. Across 20 generations and a population of 16, removing the term reversed the ordering of no pair of individuals in either catalogue, on either build (Table 3). One member of the top five changes identity in `parameterized_parts_v7`, at a boundary that falls inside a three-way fitness tie — the kind of change that cannot alter which individuals selection retains.

Catalogue	Build	σ(fitness)	Δmfg	r	Discordant	top-5	top-10
`primitives`	v0.4.0	1.0909	0.0186	0.0009	0 / 120	0	0
`primitives`	repaired	1.2737	0.2536	0.0100	0 / 120	0	0
`parameterized_parts_v7`	v0.4.0	0.6043	0.0692	0.0057	0 / 120	0	0
`parameterized_parts_v7`	repaired	0.1637	0.4322	0.1320	0 / 120	1	0

Table 3 Counterfactual removal of the manufacturability term within a single 20-generation run (population 16, seed 42, speed task), on two builds of the same repository. v0.4.0 rows: `experiments/ablation_manufacturability/results_{catalogue}.json` at tag `v0.4.0`. Repaired rows: `results_v2_{catalogue}_s42.json` at commit `a246c14`. σ is the population standard deviation of the stored fitness vector and Δmfg the within-population range of the stored score; $r = w \cdot \Delta mfg/\sigma$ at $w = 0.05$. A discordant pair is a pair whose order genuinely reverses; order changes inside a tie are not counted, because they cannot change which individuals selection keeps. Both v0.4.0 spreads lie below the 0.20 maximum that the shipped scoring function could produce (Section 7.3); the two repaired spreads are larger because they are what the repair was for.

A naive rank statistic reads this differently: counting positions after re-sorting gives 2 of 16 individuals moved in `primitives` and 8 of 16 in `parameterized_parts_v7` on the shipped build. Inspection of the sorted fitness values shows why those counts are misleading. The final populations contain only nine distinct fitness values, and 31–38% of individuals sit at exactly 0.0, so a large share of "rank changes" are permutations inside a tie — permutations that selection resolves by identifier, not by score. The discordant-pair count, which excludes ties, is the quantity that answers the question actually being asked, and it is zero on all four rows. We report this because the tie structure is itself an independent property of the release: a population in which a third of individuals share the score 0.0 has limited resolution for any added objective term, however well designed.

7.3 Two instrumentation defects

A null result invites the question of whether the instrument was working. It was not, in two independent ways.

D1 — the scoring function had no gradient. The five checks (build volume, wall thickness, assembly complexity, torque budget, cantilever) were each written as a threshold test with a capped penalty. A part either passed, losing nothing, or failed, losing a fixed amount, so the theoretical range of the composite score was [0.80, 1.00] and in practice almost nothing triggered. Sampling 100 randomly generated bodies from each of the 21 shipped catalogues, the score was *identically constant* for 7 of the 21 — no body differed from any other body. A further defect sat inside the torque check: when a body had no actuated joints the check returned 1.0, so a morphology that cannot move under its own power received full marks for its torque budget. Because no combination of checks could move the composite by more than 0.20, no two individuals could ever be separated by a manufacturability contribution larger than $0.05 \times 0.20 = 0.01$, which is why the v0.4.0 rows of Table 3 report spreads of 0.019 and 0.069 and no reordering.

D2 — the weight was never connected. `aggregate_fitness_generic` obtained the manufacturability weight through `get_weight`, which reads `reward.components` — the namespace that configures the reinforcement-learning reward. The task files declared `manufacturability` in `fitness_components` and `fitness_formula` instead, and `fitness_formula` was used only to compose a description string. The lookup therefore always missed, and the aggregator silently fell back to a hard-coded 0.05. The weights written in the task files — 0.03 for `multi_objective`, 0.01 for `speed_density` — never took effect. This is the more consequential of the two defects, because it is invisible: a declared weight that is silently replaced by a default produces a run that looks correct and is not.

7.4 Repair

Both defects are fixed in the release accompanying this paper.

- All five checks are now graded rather than binary: a design threshold maps to a score of 0.5, with headroom above it and progressive loss below it. The torque check reports *not applicable* for bodies without hinge joints and is dropped from the weighted sum, which is then renormalised over the applicable checks; a body with actuated joints and zero available torque now scores 0.0 rather than 1.0.
- A separate `fitness_weights` namespace is introduced, distinct from the reward configuration. The aggregator resolves weights in the order explicit fitness weight, then reward component, then fallback — and the fallback emits a warning instead of passing silently.
- The internal weights are redistributed according to the resolving power of each check: assembly complexity 0.35, torque budget 0.25, cantilever 0.15, wall thickness 0.15, build volume 0.10.

The effect on the instrument is measurable. Repeating the same sampling procedure (21 catalogues × 100 bodies) gives (Table 4):

Quantity	v0.4.0	Repaired
Cross-catalogue mean range	0.9760 – 1.0000	0.8147 – 0.9838
Mean range within catalogue	0.0240	0.2254
Catalogues with zero range	7 / 21	0 / 21
`primitives` range	0.0000	0.0958
`parameterized_parts_v7` range	0.0857	0.4627

Table 4 Dynamic range of the fitness-side manufacturability score before and after repair, measured on 100 randomly generated bodies per catalogue for all 21 shipped catalogues. The mean within-catalogue range grows by a factor of 9.4, and no catalogue is constant after the repair.

A gradient, however, is not the same thing as influence, and it is worth being precise about why. The score now discriminates between bodies. Whether that discrimination reaches selection depends on how its spread compares with the spread of the rest of the objective, which the repair does not change.

7.5 A bound on reordering, and the statistic that makes it usable

The bound. Write f for fitness including the term, m for the score, and `g = f − w·m` for the counterfactual. A pair (i, j) reverses only if `sign(f_i − f_j) ≠ sign(g_i − g_j)`; since `g_i − g_j = (f_i − f_j) − w(m_i − m_j)`, that requires `|f_i − f_j| < w·|m_i − m_j| ≤ w·Δmfg`. Reversals are therefore a subset of the pairs whose fitness gap is smaller than the term's maximum contribution. The size of that subset, as a fraction of all pairs, is an upper bound on the discordant fraction. The bound is exact rather than asymptotic, it requires one generation and no counterfactual run, and it is the quantity a practitioner can compute before committing to a weight.

The single-generation statistic. We call that fraction the *at-risk fraction* `A(w)`. It is tighter than the range-based form, because `|m_i − m_j|` is replaced by the actual pairwise difference rather than its maximum: at `w = 0.05` on `primitives`, 19 of 120 pairs fall below `w·Δmfg` but only 7 fall below `w·|m_i − m_j|`. The ratio `r = w·Δmfg/σ(fitness)` orders the two catalogues correctly and is the cheapest summary of the same information, but it is a proxy; the sweep below shows why it should not be read as a threshold.

Catalogue	w	r	At-risk pairs	Discordant	top-5	top-10
`primitives`	0.05	0.010	7 / 120	0	0	0
`primitives`	0.2	0.040	9 / 120	1	0	0
`primitives`	1.0	0.199	10 / 120	2	0	0
`primitives`	2.0	0.398	15 / 120	2	0	0
`parameterized_parts_v7`	0.05	0.132	14 / 120	0	1	0
`parameterized_parts_v7`	0.2	0.528	18 / 120	3	1	0
`parameterized_parts_v7`	0.5	1.320	32 / 120	13	2	0
`parameterized_parts_v7`	1.0	2.640	57 / 120	27	3	1

Table 5 Weight sweep on the repaired build, seed 42. *At-risk pairs* is the number of the 120 possible pairs whose fitness gap is smaller than `w·|mfg_i − mfg_j|`; by the bound above, the discordant count can never exceed it, and it does not in any row. Top-5 and top-10 counts refer to set membership, not to order. The two catalogues differ by a factor of 7.8 in fitness standard deviation and do not share a threshold: `primitives` reverses its first pair at `w = 0.2` and its last at `w = 1.0`, while `parameterized_parts_v7` is still intact at `w = 0.1` and has lost 27 pairs at `w = 1.0`.

A correction to our own earlier reading. An earlier draft of this section reported a common onset of reordering near `r ≈ 0.4–0.5`, inferred from the two catalogues' sweeps. The exact count does not support that reading: `primitives` reverses its first pair at `r = 0.04`, an order of magnitude below the claimed threshold, and yet never exceeds 2 of 120 pairs within the swept range, while `parameterized_parts_v7` is still intact at `r = 0.26`. The ratio orders the two catalogues correctly but does not calibrate them to a shared threshold, because the same σ can describe very different gap structures — `primitives` has six zero gaps, one gap of 1.02 and a median non-zero gap of 0.37, so its σ is inflated by a handful of outliers. The at-risk fraction is the quantity with the guarantee; the ratio is a screening summary of it. We state the correction because the superseded claim is precisely the one a reader would carry away.

Where the bound applies. The inequality holds for any scalarised objective and any weight, so it needs no assumption about the metric's design. Its *usefulness* is narrower: it is informative only while `A(w)` is small, it evaluates a single generation of the population that selection will actually act on, and it says nothing about steady-state or archive-based selection, where the surviving set is not produced by one ranking. It also cannot say whether a reversal is beneficial — only whether it is possible. It is a screening device for deciding whether a weight is worth running, not a design rule that a weight can be read off.

7.6 Why a weighted sum rather than a third objective

Why the release scalarises. The aggregator composes a scalar fitness, and every search operator in the release consumes that scalar: the generational loop, the NSGA-II/III front construction and the MAP-Elites archive all rank individuals by it. Adding manufacturability as a weighted component is the only change that leaves those operators untouched, which is why the release was built that way. The choice is nevertheless not neutral for the result, and the stored populations say by how much.

Quantifying the alternative. Treating fitness and score as two objectives — the minimal form of the third-objective proposal — leaves 3 of 16 individuals non-dominated on the `primitives` population, and the most manufacturable individual (`mfg = 1.000`) ranks eighth of sixteen in fitness. A weighted sum at `w = 0.05` cannot retain that individual; a Pareto rule retains it by construction. On `parameterized_parts_v7` the front collapses to a single individual, because the fittest individual happens also to be the most manufacturable. The two catalogues therefore differ in kind and not only in degree: the one in which the weighted sum fails most visibly is also the one in which a Pareto rule would act most strongly.

What this means for the result. Sections 7.2 and 7.7 measure the behaviour of a weighted sum. They do not measure the informativeness of the score, and the distinction matters: under a Pareto rule the term would act on the surviving set with no weight at all, and the replication problem of Section 7.7 would disappear along with the weight. We did not run that comparison and we do not claim it would favour the term — on a sixteen-individual population a two-objective front of three members is itself a small and unstable object. The experiment that would settle it is specific: the same two catalogues and three seeds, `nsga2` with objectives (task fitness, manufacturability, behavioural diversity), reporting the distribution of manufacturability along the final front and the same discordance statistics. Until it is run, the negative result of this section is properly read as a statement about scalarisation.

7.7 Does crossing the bound improve the population?

The bound concerns a single generation. A separate claim would be that choosing a weight large enough to reorder individuals causes evolution to produce more manufacturable populations over many generations. We tested that claim directly, because it does not follow from the bound and it is the claim a reader would actually want.

Using `parameterized_parts_v7` and the speed task, we ran arms at w = 0.05 (baseline, r = 0.13 at seed 42) and w = 0.50 (treatment, r = 0.24–1.32) for 20 generations at population 16, over three seeds. Because the evaluation workers do not inherit the parent seed (Section 7.1), same-seed arms are not strictly paired, so the differences below are analysed as independent samples rather than as paired ones. Each arm takes about fifteen minutes, and runs are serial because concurrent evaluation exceeds available memory. The measured quantity is the mean manufacturability of the final population (Table 6).

Seed	Baseline mean	Treatment mean	Δ mean	Δ median	Baseline mfg < 0.8	Treatment mfg < 0.8
7	0.8147	0.6394	−0.1753	−0.3024	6 / 16	11 / 16
13	0.7704	0.7834	+0.0130	+0.0875	9 / 16	8 / 16
42	0.7360	0.8049	+0.0689	+0.1300	12 / 16	8 / 16
Aggregate	—	—	**−0.0311**	−0.0283	—	—

Table 6 Verification run at w = 0.50, a weight that clears the at-risk bound on this catalogue. The realised diagnostic ratio differs by seed — 0.92, 0.42 and 1.32, because the post-run fitness spread is itself a sample — which is a further reason to treat `r` as a summary rather than

a set point. Two of three seeds move in the expected direction and one moves sharply against it; the aggregate is negative. The cross-seed range of the effect (0.244) is larger than the effect itself, and the baseline arm alone varies by 0.079 across seeds.

The effect is not detectable at this budget. Three observations support that reading rather than a reading of genuine harm. First, the sign is not consistent: two seeds improve and one degrades sharply, which is what a small, noisy effect looks like rather than what a systematic penalty looks like. Second, the variability exceeds the signal: the spread of the per-seed differences (0.244) is eight times the absolute aggregate, and the baseline arm's own seed-to-seed spread of 0.079 is comparable to the treatment effect. Third, the population is small: sixteen individuals evaluated on a stochastic physics task cannot resolve a shift of a few hundredths in a mean.

To state how much evidence would be needed, we ran a power calculation on the observed between-arm differences (two-sided $\alpha = 0.05$, 80% power, observed standard deviation of the difference 0.128): resolving an effect of 0.10 would require 13 seeds (26 runs, about 6.5 hours), 0.07 would require 27 seeds (54 runs, about 13.5 hours), 0.05 would require 52 seeds (104 runs, about 26 hours), and 0.03 would require 143 seeds. The observed effect is in the 0.03–0.07 band. The honest conclusion is therefore not that raising the weight fails, but that establishing whether it succeeds requires between one and three days of computation per question — and that this estimate is itself optimistic, because the evaluation workers do not inherit the parent seed, so same-seed arms are not strictly paired.

Reducing variance is more economical than adding seeds, and the ordering of remedies is clear from where the variance comes from: make evaluation reproducible and genuinely paired so that a seed identifies a run rather than merely a starting point; run long enough that final populations have converged, so that trajectory stochasticity stops contributing; and only then enlarge the population and add seeds.

7.8 Two different quantities share one name

`compute_manufacturability` applies the five geometric checks described above and is what the search layer consumes. `score_manufacturability` instead scores structure, materials, tolerances and cost, with the structure term driven by the number of colliding part pairs, and is what the exported report quotes. This is the origin of an inconsistency recorded in the repository documentation, where the reference body reports `overall` 0.9925 beside `manufacturability_score` 0.70 in the same file: the two numbers answer different questions. The export-side score is itself close to constant, returning exactly 0.70 for 20 of the 21 catalogues.

7.9 What does work: the export layer

The defects the objective cannot see are detected downstream, and they are the norm rather than the exception. In every run reported here, all 16 individuals in the final population contain colliding part pairs, with up to 151 colliding pairs in a single body, while the fitness-side score of those same bodies remains above 0.97. The reference body of Section 5 carries 120 such pairs and 11 out-of-range joint limits. Self-collision is therefore not a tail case that a filter would catch at the end; it is the typical state of the population, and the interference report is the only component of the framework that surfaces it.

7.10 Consequence for the contribution

The claim this section supports is narrow and, we think, more useful than the one we started with. It is not that manufacturability can be optimised by adding a weight, and not that it cannot; it is that the question is decidable before a run is spent, and that the answer depends jointly on the weight, the metric's spread and the population's gap structure. Three consequences follow. First, a term can be present in the objective, declared in the configuration and reported in every log line, and still be unable to change which individuals survive; the at-risk fraction of Section 7.5 is the cheapest test of that, and it costs one generation. Second, a metric's dynamic range is a necessary but not a sufficient repair: raising the range from 0.024 to 0.225 made the score informative without making the objective consequential. Third, the choice of scalarisation is itself an experimental variable. The weighted sum was inherited from the aggregator rather than chosen, and Section 7.6 shows that a Pareto rule would retain individuals that no weight at the reported scale can retain. What the framework contributes to manufacturing in this release is therefore the export and verification chain of Section 4 — a design is carried to STEP, DXF, bill of materials and an interference report, and the defects that search ignores are reported there — together with a measurement apparatus for the objective that the search layer consumes. Making the objective term consequential remains open; what is no longer open is how to find out whether it is.

8 Limitations of the current release

Every item below is reproducible from the repository, and each is mirrored in the repository documentation so that the software description and the paper cannot drift apart.

- **The reference body is a prototype, not an engineered design.** It contains 120 colliding part pairs and eleven joints with implausible parameter ranges (six collapsed to [0, 0], five spanning more than 4π rad). Self-collision penalties and physically meaningful joint limits are not yet part of the fitness function.
- **Control training is per-individual.** A separate trainer is constructed for each evaluated individual and updated within that individual's budget; no single policy is shared across bodies and no zero-shot transfer is claimed.

- **The repaired objective is under-powered at the reported budget.** The bound in Section 7.5 can be cleared by raising the weight, but the population-level benefit of doing so is not detectable with 16 individuals, 20 generations and three seeds; a power calculation puts the requirement at 13 to 143 seeds per comparison. Claims about how manufacturability objectives reshape evolution must therefore wait for a cheaper evaluation path, not for a better metric.
- **Population resolution is limited by ties.** Final populations contain about nine distinct fitness values, and 31–38% of individuals score exactly 0.0. Any added objective term competes for differences that the evaluator does not currently resolve, so the at-risk fraction of Section 7.5 is a bound on what a weight can achieve only insofar as the gaps it measures are real rather than quantisation artefacts.
- **The scalarisation was inherited, not chosen.** Manufacturability enters as a weighted component of a scalar fitness because that is what the aggregator and the NSGA-II/III front construction consume. Section 7.6 shows that a Pareto treatment would retain individuals no weight at the reported scale can retain, so the negative result reported here is confined to the scalarised formulation until the three-objective run described there is carried out.
- **Geometric decomposition is not manufacturing decomposition.** The parametric layer splits a body into parts that are individually printable and mutually placeable; it does not derive a decomposition from fixturing, tool access, assembly order or tolerance stack-up. A body that passes every check may therefore still require a decomposition a fabricator would not choose.
- **Manufacturability and cost aggregates coexist without reconciliation.** The two manufacturability aggregates and the three mass and two cost estimates of Table 2 come from independent code paths that nothing reconciles. The two manufacturability aggregates are distinct definitions rather than an arithmetic error (Section 7); the mass and cost discrepancies remain unexplained.
- **STEP output is tessellated.** Exports conform to AP242 but carry triangle meshes rather than analytic B-rep geometry (Section 4).
- **Degenerate joint ranges are reported, not rejected.** Range ordering is now normalised in the genetic operators and the joint optimiser, so the reference snapshot contains no inverted range; but nothing constrains how wide a range may become, and the exporter surfaces the resulting warnings rather than refusing to export, so a physically implausible limit reaches the delivered files unless the reader inspects the report.
- **The finite-element solver discretises the bounding box, not the part.** `surface_to_tetrahedralize` in `forgecraft/analysis/fea.py` derives its grid spacing from the part's bounding box and fills that box with regular volume elements; it does not tetrahedralise the part's surface geometry. Results describe a solid block of the part's outer dimensions and must not be read as a stress or safety-factor check.
- **The topology optimiser does not solve a mechanical problem.** The solid isotropic material with penalisation (SIMP) loop in `forgecraft/analysis/topology.py` never assembles or solves Ku = f; `spsolve` is imported but never called, and its sensitivity term depends only on the density field. The `compliance` field of `TopologyResult` is hard-coded to 0.0.
- **Rendering is not physically based.** Figures are rasterised through matplotlib with per-face Lambert/Blinn-Phong/Fresnel shading under a fixed three-light rig — adequate for schematic views, not a path tracer, and not intended for material comparisons.
- **No performance benchmarks are shipped, and distributed evaluation is not exercised in the reported results.** The repository contains no frames-per-second or speed-up measurements; throughput figures from early documentation drafts were removed rather than replaced with estimates. The Ray and multiprocessing paths dispatch to the same rollout-and-PPO worker used in-process and therefore agree with the single-machine path by construction, but the experiments above use the in-process evaluator.
- **No evolved design has been fabricated.** The manufacturing deliverables are generated and internally checked, but no design from this framework has been printed, assembled and tested. This is the largest gap between what the framework produces and what it demonstrates, and it is the next milestone rather than a completed result.

9 Discussion

Manufacturability as an experimental variable. The measurement in Section 7 reframes this question rather than closing it. Buildability is reported but not effectively optimised, and the reason is now specific to the point of being quantitative: a term added to a scalar objective can only reorder individuals whose fitness gaps are smaller than its own contribution, so the fraction of such pairs bounds everything any weight can achieve. That fraction was 6–8% for one catalogue across a twentyfold weight sweep and 12–48% for another, and the difference — not the weight — is what separates the two experiments. The remaining choices are engineering decisions rather than mysteries: compute the at-risk fraction, decide whether the reachable reordering is worth the run, and if not, change the formulation rather than the metric. Once that is done the interesting question becomes available: how does the structure of evolutionary search change when designs must be buildable? Treating part standardisation, minimum feature size and assembly complexity as objectives rather than filters would then turn a post-hoc constraint into an experimental factor. The framework makes this study *possible*; we have not yet carried it out, and Section 7 is an account of why the naive version of it does not work.

Reproducible comparison and a bridge between two communities. By providing a tested, documented and permissively licensed implementation with declarative design spaces, ForgeCraft allows design spaces and tasks to be shared as

configuration files, making comparisons between methods concrete rather than approximate. Its output formats target the tools mechanical engineers already use — STEP for CAD interchange, DXF for drawing review, bills of materials for procurement — so an evolved design can enter a mechanical workflow without being re-modelled from scratch, subject to the tessellation caveat above. The full pipeline also runs on commodity laptops with no proprietary software, which supports teaching co-design as an end-to-end engineering activity — from fitness function to drawing sheet — rather than as an isolated optimisation exercise.

10 Conclusion

ForgeCraft couples morphology–control co-evolution with a manufacturing-aware export chain, so that evolved robot designs are delivered together with the geometry, drawings, tolerances and documentation needed to build them. It integrates established components — graph-based morphology representation, GNN-encoded PPO control, quality-diversity and multi-objective search, MuJoCo physics — and adds an explicit representation of evolved bodies as parametric mechanical parts, together with an export and verification chain that reports the defects the search objective does not see. The paper's contribution is a measurement rather than an algorithm. Measured on two builds of the release, the manufacturing term was inert in the shipped one because the scoring function had no gradient and its weight was never read from the task file; after repair the score discriminates, but selection still does not respond, because a scalarised term can only reorder pairs whose fitness gaps are smaller than its own contribution. That condition is an exact bound, computable from a single generation, and it explains why the same nominal weight behaves differently in two catalogues. Whether clearing the bound improves evolved populations is not resolvable at three seeds; the power analysis puts the requirement at 13 to 143. ForgeCraft is distributed under the MIT licence with 424 automated tests and continuous integration, and is intended as reusable infrastructure for research at the boundary between evolutionary robotics and design for manufacturing.

Future work is ordered by dependency. First, the search objective needs the terms it currently lacks — self-collision penalties and physically meaningful joint limits — and the mass and cost models must be unified into a single reconciled source. Second, the empirical questions the framework exists to answer come into reach: how manufacturability constraints reshape evolved morphologies, and how search behaves when buildability is an objective rather than a filter. Third, and decisively, the loop must be closed in hardware: fabricating and testing an evolved design is the only evidence that will establish whether the manufacturing stage reported here is a real capability or an elaborate serialisation format. We state the scope of this release plainly: it is a research prototype whose export chain is functional and exercised end to end, and whose search chain is now free of the defect that silenced its locomotion signal but still optimises an objective that permits self-intersecting bodies. The reference example is a pipeline-completeness demonstration, not a motile design.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. Zhenxin Liu — Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Data and code availability. The framework is released under the Massachusetts Institute of Technology (MIT) licence at https://github.com/jige0611/forgecraft. The reference run is committed in full under `examples/demo_robot/`; STL, STEP, DXF, URDF and the production archive regenerate from the committed `best_body.json`. The shipped release measured in Section 7.1 is https://github.com/jige0611/forgecraft/releases/tag/v0.4.0; the repair described in Sections 7.2–7.3 and the experiments of Sections 7.4–7.5 are on the default branch of the same repository and are not part of that tag.

Use of generative artificial intelligence (AI). Generative AI tools, specifically the Trae coding assistant, were used for code assistance, experiment scripting and language editing during preparation of this manuscript and the associated software. All scientific claims, results and references were verified by the author, who takes full responsibility for the content.

References

1. Sims K (1994) Evolving virtual creatures. In: Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '94), pp 15–22. https://doi.org/10.1145/192161.192167
2. Zhao A, Xu J, Konaković-Luković M, Hughes J, Spielberg A, Rus D, Matusik W (2020) RoboGrammar: graph grammar for terrain-optimized robot design. ACM Transactions on Graphics 39(6):188:1–188:16. https://doi.org/10.1145/3414685.3417831
3. Gupta A, Savarese S, Ganguli S, Fei-Fei L (2021) Embodied intelligence via learning and evolution. Nature Communications 12(1):5721. https://doi.org/10.1038/s41467-021-25874-z
4. Yuan Y, Song Y, Luo Z, Sun W, Kitani K. M. (2022) Transform2Act: learning a transform-and-control policy for efficient agent design. In: International Conference on Learning Representations (ICLR 2022). Preprint arXiv:2110.03659
5. Pathak D, Lu C, Darrell T, Isola P, Efros A. A. (2019) Learning to control self-assembling morphologies: a study of generalization via modularity. In: Advances in Neural Information Processing Systems 32 (NeurIPS)
6. Mouret J-B, Clune J (2015) Illuminating search spaces by mapping elites. arXiv:1504.04909

7. Pugh J. K., Soros L. B., Stanley K. O. (2016) Quality diversity: a new frontier for evolutionary computation. Frontiers in Robotics and AI 3:40. https://doi.org/10.3389/frobt.2016.00040
8. Bhatia J. S., Jackson H, Tian Y, Xu J, Matusik W (2021) Evolution Gym: a large-scale benchmark for evolving soft robots. In: Advances in Neural Information Processing Systems 34 (NeurIPS 2021). Preprint arXiv:2201.09863
9. Hupkes E, Jelisavcic M, Eiben A. E. (2018) Revolve: a versatile simulator for online robot evolution. In: Applications of Evolutionary Computation (EvoApplications 2018), Lecture Notes in Computer Science 10784. Springer, Cham, pp 687–702. https://doi.org/10.1007/978-3-319-77538-8_46
10. Duarte M, Silva F, Rodrigues T, Oliveira S. M., Christensen A. L. (2014) JBotEvolver: a versatile simulation platform for evolutionary robotics. In: Proceedings of the 14th International Conference on the Synthesis and Simulation of Living Systems (ALIFE 14). MIT Press, Cambridge, Massachusetts, pp 210–211. https://doi.org/10.7551/978-0-262-32621-6-CH035
11. Gibson I, Rosen D, Stucker B, Khorasani M (2021) Additive Manufacturing Technologies, 3rd edn. Springer, Cham. https://doi.org/10.1007/978-3-030-56127-7
12. Boothroyd G, Dewhurst P, Knight W. A. (2010) Product Design for Manufacture and Assembly, 3rd edn. CRC Press, Boca Raton. https://doi.org/10.1201/9781420089288
13. ISO (2022) ISO 10303-242:2022 Industrial automation systems and integration — product data representation and exchange — part 242: application protocol: managed model-based 3D engineering, 3rd edn. International Organization for Standardization, Geneva
14. ISO (1989) ISO 2768-1:1989 General tolerances — part 1: tolerances for linear and angular dimensions without individual tolerance indications, 1st edn. International Organization for Standardization, Geneva
15. Brockman G, Cheung V, Pettersson L, Schneider J, Schulman J, Tang J, Zaremba W (2016) OpenAI Gym. arXiv:1606.01540
16. Gilmer J, Schoenholz S. S., Riley P. F., Vinyals O, Dahl G. E. (2017) Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research (PMLR) 70:1263–1272
17. Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. In: International Conference on Learning Representations (ICLR 2018). Preprint arXiv:1710.10903
18. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. arXiv:1707.06347
19. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 6(2):182–197. https://doi.org/10.1109/4235.996017
20. Deb K, Jain H (2014) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. IEEE Transactions on Evolutionary Computation 18(4):577–601. https://doi.org/10.1109/TEVC.2013.2281535
21. Todorov E, Erez T, Tassa Y (2012) MuJoCo: a physics engine for model-based control. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp 5026–5033. https://doi.org/10.1109/IROS.2012.6386109